

PT2 Slides and Final Project Summary

Sources reviewed

- `pt2-slides/IDD\_CLASS1\_BSE\_JUNE2026.pdf`
- `pt2-slides/IDD\_CLASS2\_BSE\_JUNE2026.pdf`
- `pt2-slides/IDD\_CLASS3\_BSE\_JUNE2026.pdf`
- `pt2-slides/IDD\_CLASS4\_BSE\_JUNE26.pdf`
- `pt2-slides/IDD\_CLASS5\_BSE\_JUNE26.pdf`
- `project-guidelines/IDD\_ASSIGNMENT.pdf`
- `project-guidelines/final\_project.py`

Executive summary

Across the slide decks, the course moves from a broad idea to a deployable cloud project. The core message is that strong data science is not enough on its own. A useful intelligent application must also be planned well, built collaboratively, versioned, tested, deployed to the cloud, monitored in production, and explained in business terms.

That message maps directly to the final Glovo forecasting project. The assignment is not only asking for a model with good accuracy. It is asking for an end-to-end forecasting solution that can run in AWS, produce the required prediction file, and show that the team understands how Agile, DevOps, DataOps, MLOps, and cloud architecture support a real operational use case.

Slide deck summary

Class 1: course framing, industry shift, and Agile foundations

This deck opens by explaining why Intelligent Data Driven Applications matters. The instructor frames the course as a bridge from notebook-based analysis to production-ready AI-enabled applications. The slides emphasize that the rise of AI agents and copilots increases the need for engineering discipline rather than replacing it. Clear requirements, good data quality, version control, testing, cloud architecture, monitoring, and human judgment remain essential.

The deck then introduces the final Glovo project at a high level. It explains the business problem of forecasting hourly order volume so operations can plan courier capacity. It also states the key deliverables: cloud-runnable code, a short video, a predictions CSV stored in S3, and a short production-considerations PDF.

The second half of the deck moves into Agile. It contrasts Waterfall with Agile and explains why Agile fits data science projects better: data work is exploratory, requirements evolve, and stakeholder feedback matters. Scrum and Kanban are introduced as practical delivery frameworks. The slides cover Scrum roles, ceremonies, artifacts, and the meaning of "definition of done," along with Kanban as a lighter-weight workflow for continuous work.

Main lesson from Class 1: a data science project should be run like a product, not like a one-off analysis. The team should organize work in small increments, define ownership clearly, and keep a shared view of what "done" means.

Class 2: DevOps, DataOps, and MLOps for forecasting systems

This deck explains why a good model is not automatically a usable business solution. It uses forecasting examples to show the gap between building a model and operating it reliably every week. The DevOps section focuses on the software delivery lifecycle: plan, code, build, test, release, deploy, operate, and monitor. It also highlights common tooling such as Git, CI/CD platforms, workflow orchestration tools, monitoring tools, and container or API deployment options.

The deck then introduces DataOps as DevOps for data pipelines. The slides focus on data freshness, schema validation, row-count checks, anomaly checks, orchestration, and environment movement from development to production. The working-session examples are especially relevant to forecasting: missing weather data, empty promotions data, schema changes, late-arriving files, and suspicious spikes in volume.

The MLOps section adds model-specific concerns. It explains that reproducibility depends on both code versions and data versions. It covers maturity levels, team personas, lifecycle stages, deployment approaches, drift monitoring, retraining logic, and rollback. The final message is that an ML system must monitor both pipeline health and forecast quality after deployment.

Main lesson from Class 2: to productionize forecasting, you need three layers working together. DevOps handles software delivery, DataOps protects data reliability, and MLOps protects model reliability.

Class 3: shortened continuation of DataOps and MLOps

The extracted text from this file overlaps heavily with the DataOps and MLOps material from Class 2 and appears to be a shorter continuation or condensed version of that content. It repeats the core ideas around data-pipeline controls, monitoring, orchestration, model lifecycle management, environment strategy, and production troubleshooting.

Because of that overlap, the practical takeaway is reinforcement rather than a brand-new topic. The course is signaling that data quality checks, retraining strategy, traceability, and role clarity are not optional extras. They are part of the expected design for a real ML application.

Main lesson from Class 3: the forecasting solution should be designed to survive operational change, not just produce one good backtest.

Class 4: cloud computing concepts and AWS building blocks

This deck shifts from process to infrastructure. It explains what cloud computing is, why cloud adoption has accelerated, and how public cloud differs from traditional data centers. The slides emphasize lower upfront cost, elastic scaling, faster provisioning, and the shift from owning infrastructure to consuming managed services.

The deck covers cloud deployment models: public, private, and hybrid cloud. It then focuses on AWS services, especially storage and compute. S3 is presented as the main object store, and Lambda is introduced as an event-driven compute option. The deck also transitions into a hands-on section about deploying a forecasting workflow on AWS using IAM users, S3, Lambda, and CloudFormation.

This is the strongest cloud-to-project bridge in the lecture material. It shows that even a simple forecasting pipeline can be represented as cloud components with permissions, storage, code execution, and infrastructure-as-code.

Main lesson from Class 4: the final project should not just "use AWS" loosely. It should show a deliberate mapping from business workflow to cloud services such as S3, Lambda, IAM, and deployment templates.

Class 5: hands-on AWS lab and operational setup

This deck is a practical lab. It walks through logging into AWS with an IAM user, uploading data to S3, creating a Lambda function, attaching permissions, using pandas inside Lambda, and writing processed output back to S3. It also includes a sample Lambda script and points to CloudFormation as a way to launch the setup more systematically.

The extractable text in the final pages is limited, but the visible structure suggests a transition from the hands-on lab to real-world use case examples and guest speakers. Even without all speaker content visible in text extraction, the intent is clear: the class connects the AWS mechanics to how production teams actually operate intelligent systems in a company setting.

Main lesson from Class 5: your team should be able to explain not only the model, but also how the cloud job is triggered, what permissions it needs, where inputs live, where outputs are written, and how failures would be handled.

How the slides connect to project-guidelines

What `final_project.py` clarifies

The marimo notebook in `project-guidelines/final_project.py` turns the assignment into a more explicit checklist. It spells out:

- The target is hourly `orders`.
- The forecast horizon is one full week, or 168 hours.
- The concrete submission window is `2022-01-24 00:00:00` through `2022-01-30 23:00:00`.
- The required output columns are `time` and `preds`.
- Validation should mimic the real production setup: forecast the next week every Sunday and avoid leakage.
- Teams should compare multiple model families, include a naive baseline, and report both MSE and SMAPE.
- `check_output_format.py` should be used to confirm that the output CSV is valid.

This file makes the technical modeling requirements much clearer than the short assignment PDF alone.

What `IDD_ASSIGNMENT.pdf` adds

The assignment PDF adds the delivery and environment context:

- The project must run in the cloud.
- The output predictions file must be stored in S3.
- There is an AWS classroom account structure with naming conventions for S3 buckets and Lambda functions.
- The short PDF deliverable must describe teams, roles, estimation, architecture, data flow, environments, build and monitoring approach, end-user interaction, and success KPIs.

This means the assignment is graded on both forecasting quality and solution design maturity.

Direct tie-in: slide concepts mapped to the final project

Agile and team process

The project PDF explicitly asks for team roles, interactions, ways of working, and rough estimates. That requirement comes straight from the Agile material in Class 1. A strong submission should explain who owns modeling, AWS infrastructure, testing, and presentation work, and how the team coordinated decisions.

DevOps

The slides say a model is not enough unless it can run automatically, be tested before release, and be maintained by someone else later. In this project, that means your forecasting code should be versioned, runnable without manual notebook tweaks, and packaged so it can execute in AWS.

DataOps

The slides repeatedly stress data freshness, schema validation, row-count checks, and orchestration. In your project, this should appear in the architecture and production-considerations writeup. Even if the classroom dataset is static, you should describe what checks would run before forecasting, what happens if input data is late or malformed, and how alerts would work.

MLOps

The assignment asks how the model will be built and monitored. That is pure MLOps territory. Your report should explain model training, validation strategy, champion-model selection, output checks, monitoring KPIs, retraining triggers, and how you would detect drift or forecast degradation.

Cloud architecture

Classes 4 and 5 show the expected AWS vocabulary for the course: IAM, S3, Lambda, and CloudFormation. A good project architecture does not need to be complicated, but it should be coherent. For example, raw data can live in S3, a training/inference job can run in Lambda or another AWS compute option, outputs can be written back to S3, and monitoring/logging can be handled through AWS-native services.

What a strong final submission should show

Based on the slides and project-guidelines files together, a strong final submission should demonstrate five things:

- Solid time-series modeling discipline: EDA, baselines, leakage-safe validation, and clear metric comparison.
- Reproducibility: code structure, dependency management, and output format checks.
- Cloud deployment thinking: inputs, compute, outputs, permissions, and automation.
- Operational thinking: data checks, monitoring, alerts, failure paths, and retraining logic.
- Business framing: why the forecast matters, who uses it, and how success is measured.

Suggested project narrative for your team

If you want your final presentation and PDF to feel closely aligned with the course, the cleanest narrative is:

- 1. We translated the Glovo operations problem into a weekly forecasting task.
- 2. We explored the data and selected a model using leakage-safe validation and business-relevant metrics.
- 3. We packaged the forecasting workflow so it can run in AWS and write predictions to S3.
- 4. We designed the surrounding controls using DevOps, DataOps, and MLOps ideas from class.

- 5. We defined how the system would be monitored, maintained, and used by operations.

That story matches the exact progression of the lecture materials.

Bottom line

The slide decks are not separate from the project; they are the rubric behind it. Class 1 explains how to organize the work. Classes 2 and 3 explain how to make the data and model production-ready. Classes 4 and 5 explain how to map the solution into AWS. The project-guidelines files then turn those course ideas into a concrete forecasting deliverable with strict output requirements and a cloud deployment expectation.